<u>Práctica de laboratorio 2</u>

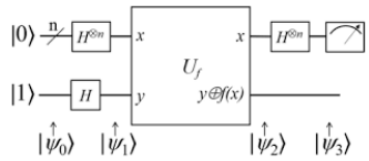
1. Deutsch-Jozsa

a. Demostrar que, dado un oráculo U_f y su funcionamiento, entonces $U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle$.

En el algoritmo de Deutsch-Jozsa, el oráculo asociado a una función booleana $f:\{0,1\}^n \to \{0,1\}$. Actúa sobre dos registros de la siguiente forma: $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$

donde $|x\rangle$ es el registro de entrada, $|y\rangle$ es un qubit auxiliar y $\oplus$ significa suma módulo 2, o XOR.

El oráculo no cambia el registro $|x\rangle$, solamente modifica el segundo qubit dependiendo del valor de $f(x)$.



Partamos de $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$, entonces el estado de entrada al oráculo es:

$$|x\rangle|-\rangle = |x\rangle\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = \frac{1}{\sqrt{2}}(|x\rangle|0\rangle - |x\rangle|1\rangle)$$

Aplicando U_f: $U_f|x\rangle|-\rangle = \frac{1}{\sqrt{2}}\left(U_f|x\rangle|0\rangle - U_f|x\rangle|1\rangle\right)$

$$\begin{cases} U_f|x\rangle|0\rangle = |x\rangle|0 \oplus f(x)\rangle \\ U_f|x\rangle|1\rangle = |x\rangle|1 \oplus f(x)\rangle \end{cases} \Rightarrow U_f|x\rangle|-\rangle = \frac{1}{\sqrt{2}}(|x\rangle|0 \oplus f(x)\rangle - |x\rangle|1 \oplus f(x)\rangle)$$

Como $f(x)$es una función booleana, solo puede valer: $f(x) = 0,\ f(x) = 1$. Entonces hacemos los dos casos.

<u>Caso 1: $f(x) = 0$</u>

Si $f(x) = 0$, entonces: $\begin{cases} 0 \oplus f(x) = 0 \oplus 0 = 0 \\ 1 \oplus f(x) = 1 \oplus 0 = 1 \end{cases}$, $\therefore U_f|x\rangle|-\rangle = |x\rangle\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |x\rangle|-\rangle$.

Además, se observa que, si $f(x) = 0$, $(-1)^{f(x)} = (-1)^0 = 1$. Entonces este resultado también puede escribirse como:

$$U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle$$

<u>Caso 2: $f(x) = 1$</u>

Si $f(x) = 1$, entonces: $\begin{cases} 0 \oplus f(x) = 0 \oplus 1 = 1 \\ 1 \oplus f(x) = 1 \oplus 1 = 0 \end{cases}$, $\therefore U_f|x\rangle|-\rangle = |x\rangle\frac{1}{\sqrt{2}}(|1\rangle - |0\rangle) = -|x\rangle|-\rangle$.

Además, se observa que, si $f(x) = 1$, $(-1)^{f(x)} = (-1)^1 = -1$. Entonces el resultado también puede escribirse como:

$$U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle$$

■

b. Quiero que piensen en formas creativas y diferentes de poder aplicar distintos oráculos balanceados. Quiero que piensen en 3 formas que no sean precisamente intuitivas y comprueben si funcionan en D-J

<u>Oráculo balanceado 1</u>

Tomemos la función $f_1(x_1, x_2, x_3) = x_1 \oplus (x_2 x_3)$ y comprobemos explícitamente cómo funciona dentro del algoritmo Deutsch–Jozsa para $n = 3$. El algoritmo inicia con el estado $|000\rangle|1\rangle$, donde los primeros tres qubits son el registro de entrada y el último es el qubit auxiliar.

Aplicamos Hadamard a los cuatro qubits. En el registro de entrada y en el auxiliar,

$$H^{\otimes 3}|000\rangle = \frac{1}{\sqrt{8}} \sum_{x \in \{0,1\}^3} |x\rangle, \qquad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle$$

Entonces el estado total queda

$$\frac{1}{\sqrt{8}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle)|-\rangle$$

Ahora aplicamos el oráculo. Así que necesitamos evaluar $f_1(x)$ en cada una de las 8 entradas.

x_1	x_2	x_3	$f_1(x_1, x_2, x_3)$	$(-1)^{f_1(x_1 x_2 x_3)}$
0	0	0	0	$+1$
0	0	1	0	$+1$
0	1	0	0	$+1$
0	1	1	1	-1
1	0	0	1	-1
1	0	1	1	-1
1	1	0	1	-1
1	1	1	0	$+1$

Entonces, después del oráculo, el estado queda:

$$\frac{1}{\sqrt{8}}|000\rangle + |001\rangle + |010\rangle - |011\rangle - |100\rangle - |101\rangle - |110\rangle + |111\rangle|-\rangle$$

Cuando aplicamos las Hadamard finales, la amplitud del estado $|000\rangle$ resulta ser $\frac{1}{2^n}\sum_x (-1)^{f(x)}$ y para $n = 3$:

$$\frac{1}{8}\sum_x (-1)^{f(x)} = \frac{1}{8}(1 + 1 + 1 - 1 - 1 - 1 - 1 + 1) = 0$$

Por lo tanto:

- si la suma $\neq 0 \rightarrow$ la función es constante
- si la suma $= 0 \rightarrow$ la función es balanceada

Solo revisamos 000 porque esa amplitud ya distingue completamente los dos casos en D–J. Por lo tanto, la función es balanceada.

<u>Oráculo balanceado 2</u>

Tomemos la función $f_2(x_1, x_2, x_3) = (x_1 \vee x_2) \oplus x_3$. El estado inicial es otra vez $|000\rangle|1\rangle$.

Después de aplicar Hadamard a todos los qubits, el estado se convierte en:

$$\frac{1}{\sqrt{8}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle)|-\rangle$$

Ahora aplicamos el oráculo,

x_1	x_2	x_3	$f_2(x_1, x_2, x_3)$	$(-1)^{f_2(x_1x_2x_3)}$
0	0	0	0	$+1$
0	0	1	1	-1
0	1	0	1	-1
0	1	1	0	$+1$
1	0	0	1	-1
1	0	1	0	$+1$
1	1	0	1	-1
1	1	1	0	$+1$

Entonces, después del oráculo, el estado queda:

$$\frac{1}{\sqrt{8}}(|000\rangle - |001\rangle - |010\rangle + |011\rangle - |100\rangle + |101\rangle - |110\rangle + |111\rangle)|-\rangle$$

De nuevo, para comprobar el funcionamiento en D–J calculamos explícitamente la amplitud del estado $|000\rangle$ tras las Hadamard finales:

$$\frac{1}{8}\sum_{x}(-1)^{f_2(x)} = \frac{1}{8}(1 - 1 - 1 + 1 - 1 + 1 - 1 + 1) = 0$$

Eso quiere decir que al medir el registro no se obtiene $|000\rangle$, y el algoritmo identifica que la función no es constante.

<u>Oráculo balanceado 3</u>

Tomemos la función $f_3(x_1, x_2, x_3) = x_1 \oplus x_2 \oplus (x_1x_3)$. El estado inicial es $|000\rangle|1\rangle$. Después de aplicar Hadamard a los cuatro qubits:

$$\frac{1}{\sqrt{8}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle)|-\rangle$$

Aplicamos el oráculo:

x_1	x_2	x_3	$f_3(x_1, x_2, x_3)$	$(-1)^{f_3(x_1x_2x_3)}$
0	0	0	0	$+1$
0	0	1	0	$+1$
0	1	0	1	-1
0	1	1	1	-1
1	0	0	1	-1
1	0	1	0	$+1$
1	1	0	0	$+1$
1	1	1	1	-1

Entonces, después del oráculo, el estado queda:

$$\frac{1}{\sqrt{8}}(|000\rangle + |001\rangle - |010\rangle - |011\rangle - |100\rangle + |101\rangle + |110\rangle - |111\rangle)|-\rangle$$

Calculamos la amplitud de $|000\rangle$:

El tercer oráculo también funciona correctamente como oráculo balanceado con D-J.

2. PennyLane.

a. Crea un circuito sencillo en PennyLane y busca distintos tipos de draw. Y me expliques cuál es tu favorito y por qué.

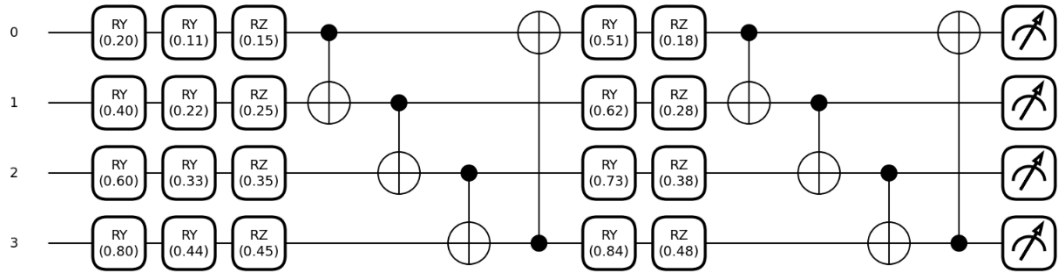
Nota. El código viene anexado al final del documento.

El circuito que escogí es uno que he utilizado mucho para algoritmos cuánticos variacionales (en este caso lo hice estático), utiliza 4 qubits y está organizado en dos bloques de rotaciones seguidos por capas de entrelazamiento. Primero se aplican rotaciones $RY(x_i)$ en cada qubit para preparar el estado inicial dependiente de los datos. Después se añaden rotaciones parametrizadas $RY(\theta_{0,i})$ y $RZ(\phi_{0,i})$ que introducen variaciones en la amplitud y en la fase del estado cuántico. A continuación, se aplica una cadena circular de compuertas CNOT que genera entrelazamiento entre los qubits. El circuito repite una segunda capa de rotaciones $RY(\theta_{1,i})$ y $RZ(\phi_{1,i})$ seguida de otro anillo de CNOT, permitiendo que la información se mezcle entre todos los qubits. Finalmente, se obtienen las probabilidades de medición del sistema completo. Este tipo de estructura con rotaciones locales y capas de entrelazamiento es común en circuitos variacionales utilizados en algoritmos cuánticos híbridos.

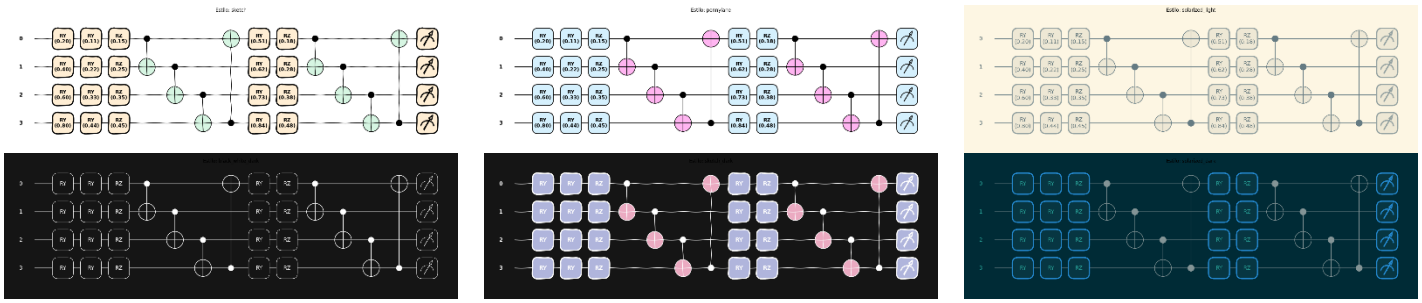
print(qml.draw(circuito_Sebas)())

```
0: ─RY(0.20)─RY(0.11)─RZ(0.15)─╭●─────────╭X─RY(0.51)─RZ(0.18)─╭●─────────╭X─╭│ Probs
1: ─RY(0.40)─RY(0.22)─RZ(0.25)─╰X─╭●──────│──RY(0.62)─RZ(0.28)─╰X─╭●──────│──│─│ Probs
2: ─RY(0.60)─RY(0.33)─RZ(0.35)────╰X─╭●───│──RY(0.73)─RZ(0.38)────╰X─╭●───│──│─│ Probs
3: ─RY(0.80)─RY(0.44)─RZ(0.45)───────╰X─╰●─│──RY(0.84)─RZ(0.48)───────╰X─╰●─│─│─│ Probs
```

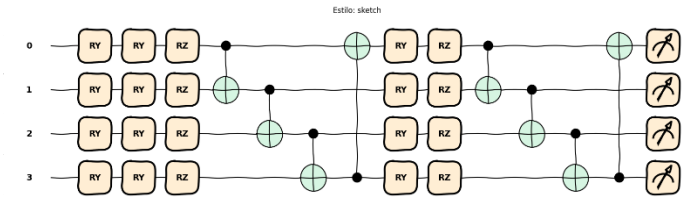
fig, ax = qml.draw_mpl(circuito_Sebas, decimals=2)()



Otros estilos.



Mi favorito sigue siendo los de Qiskit, okay sin contar eso mi favorito es el siguiente:



Me agrada mucho sus colores y su tipo de trazo como si fuese dibujado no perfecto. Considero que es muy útil para algún poster o en caso de divulgar. Hace que se vea más fácil de lo difícil que es en verdad.

b. Demostrar que los eigenvalores y eigenvectores de $\widehat{Z}$ son $+1$ con $|0\rangle$ y -1 con $|1\rangle$ y con ayuda del teorema de descomposición espectral, demostrar que el valor esperado de Z es la diferencia entre las probabilidades de medir 0 menos las de medir 1. (Hint: Expresa el operador $\widehat{Z}$ en su descomposición espectral y sustituye sobre $\langle\psi|\widehat{Z}|\psi\rangle$ para $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$)

Sea el operador de Pauli Z: $\hat{Z} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ y la base computacional $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$.

Aplicamos $|0\rangle$: $\hat{Z}|0\rangle = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = |0\rangle \Rightarrow \hat{Z}|0\rangle = +1|0\rangle, \therefore |0\rangle$ es eigenvector de $\hat{Z}$ con eigenvalor $+1$.

Aplicamos $|1\rangle$: $\hat{Z}|1\rangle = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}\begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ -1 \end{pmatrix} = -|1\rangle \Rightarrow \hat{Z}|1\rangle = -1|1\rangle, \therefore |1\rangle$ es eigenvector de $\hat{Z}$ con eigenvalor -1.

Como $\hat{Z}$ tiene eigenvalores $+1$ y -1, con eigenvectores $|0\rangle$ y $|1\rangle$, su descomposición espectral es

$$\hat{Z} = (+1)|0\rangle\langle 0| + (-1)|1\rangle\langle 1| = |0\rangle\langle 0| - |1\rangle\langle 1|$$

Ahora considerando el estado: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, con $|\alpha|^2 + |\beta|^2 = 1$. El valor esperado de $\hat{Z}$ es $\langle\hat{Z}\rangle = \langle\psi|\hat{Z}|\psi\rangle$. Sustituyendo la descomposición espectral:

$$\langle\hat{Z}\rangle = \langle\psi||0\rangle\langle 0| - |1\rangle\langle 1||\psi\rangle = \langle\psi|0\rangle\langle 0|\psi\rangle - \langle\psi|1\rangle\langle 1|\psi\rangle$$

Donde, $\langle 0|\psi\rangle = \alpha$, $\langle\psi|0\rangle = \alpha^*$, $\langle 1|\psi\rangle = \beta$ y $\langle\psi|1\rangle = \beta^*$.

$$\langle\hat{Z}\rangle = \alpha^*\alpha - \beta\beta^* = |\alpha|^2 - |\beta|^2$$

Como $P(0) = |\alpha|^2$ y $P(1) = |\beta|^2$, entonces: $\langle\hat{Z}\rangle = P(0) - P(1)$ ■

c. **Repitan el desarrollo matemático que hicimos para RX en las matrices RY y RZ, y luego generen el circuito en PennyLane, de return coloquen el valor esperado de la matriz de Paulli Z, grafíquenlo en Matplotlib y argumenten si lo que graficaron se corresponde a lo que el circuito que predijeron iba a dar. Contesta las siguientes preguntas usando tu imaginación:**
    a. **Si podemos fabricar funciones basadas en circuitos cuánticos, ¿podrían existir formas de derivar dichas funciones para obtener utilidad de las mismas?**
    b. **¿Qué nuevos horizontes el trabajar con derivadas de funciones generadas por medio de circuitos cuánticos podríamos llegar a alcanzar?**
    c. **Investiga algún algoritmo que se beneficie de ello y describe brevemente como lo hacen, e identifica que posibles avances para la humanidad podrían traer algoritmos de esta naturaleza.**

Nota: El código viene anexado.

Empezaré por el desarrollo matemático de R_Y y R_Z, las rotaciones alrededor de los ejes se definen como:

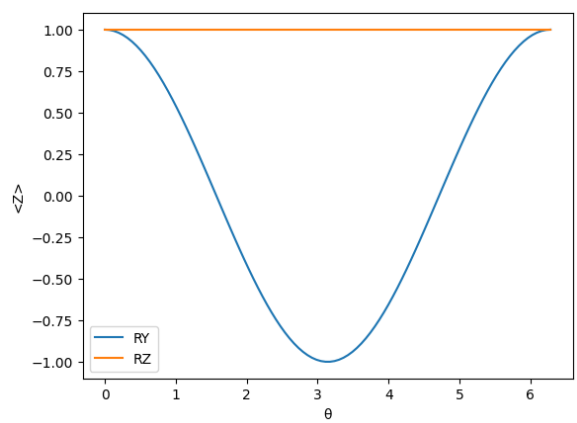
$$R_k(\theta) = e^{\frac{i\theta\sigma_k}{2}}, \text{con } \sigma_k \text{ las matrices de Pauli: } Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \text{ y } Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

$$R_Y(\theta) = e^{-i\theta Y/2} = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$$

$$R_Y(\theta)|0\rangle = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) \end{pmatrix} \Rightarrow |\psi\rangle$$

$$= \cos\left(\frac{\theta}{2}\right)|0\rangle + \sin\left(\frac{\theta}{2}\right)|1\rangle \begin{cases} P(0) = \cos^2\left(\frac{\theta}{2}\right) \\ P(1) = \sin^2\left(\frac{\theta}{2}\right) \end{cases}$$

$$\Rightarrow \langle Z\rangle = P(0) - P(1) = \cos^2\left(\frac{\theta}{2}\right) - \sin^2\left(\frac{\theta}{2}\right) = \cos(\theta)$$



Graficas de los circuitos.

$$R_Z(\theta) = e^{-i\theta Z/2} = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

$$R_Z(\theta)|0\rangle = e^{-i\theta/2}|0\rangle \Rightarrow P(0) = 1, P(1) = 0$$

$$\Rightarrow \langle Z\rangle = 1$$

c. a. Sí. Si un circuito depende de parámetros (por ejemplo, ángulos de rotación), el resultado que obtenemos del circuito también depende de ellos. Entonces se puede estudiar cómo cambia ese resultado cuando cambiamos el parámetro, es decir, calcular derivadas. Esto sirve para ajustar los parámetros del circuito y hacer que el circuito produzca el resultado que buscamos.

c. b. Esto permitiría entrenar circuitos cuánticos de forma parecida a como se entrenan modelos de aprendizaje automático. Podría ayudar a resolver problemas de optimización, estudiar sistemas físicos complicados o mejorar simulaciones en química y materiales que hoy son difíciles para las computadoras clásicas.

c. c. Un ejemplo es el VQE (Variational Quantum Eigensolver). En este algoritmo se usa un circuito cuántico con parámetros y un optimizador clásico va cambiando esos parámetros usando derivadas para encontrar la energía mínima de un sistema. Esto podría ayudar a simular moléculas y materiales, lo que podría impactar en el diseño de medicamentos, baterías o materiales más eficientes.

Computación Cuántica II

Sebastián González Juárez

Práctica de laboratorio 1.

a. Crea un circuito sencillo en PennyLane y busca distintos tipos de draw. Y me expliques cuál es tu favorito y por qué.

```
!python --version
!pip install -q --upgrade pip
!pip install -q pennylane matplotlib
```

```
Python 3.12.12
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 23.0 MB/s eta 0:00:00
```

```
import pennylane as qml
from pennylane import numpy as np
import matplotlib.pyplot as plt

print("PennyLane version:", qml.__version__)
print("Estilos disponibles:", qml.drawer.available_styles())
```

```
PennyLane version: 0.44.1
Estilos disponibles: ('black_white', 'black_white_dark', 'sketch', 'pennylane', 'pennylane_sketch', 'sketch_dark', 'solarized_li
```

```
n_qubits = 4
dev = qml.device("default.qubit", wires=n_qubits)

x = np.array([0.20, 0.40, 0.60, 0.80])
theta_0 = np.array([0.11, 0.22, 0.33, 0.44])
phi_0   = np.array([0.15, 0.25, 0.35, 0.45])

theta_1 = np.array([0.51, 0.62, 0.73, 0.84])
phi_1   = np.array([0.18, 0.28, 0.38, 0.48])

def ring_cnot():
    qml.CNOT(wires=[0, 1])
    qml.CNOT(wires=[1, 2])
    qml.CNOT(wires=[2, 3])
    qml.CNOT(wires=[3, 0])

@qml.qnode(dev)
def circuito_Sebas():
    for i in range(n_qubits):
        qml.RY(x[i], wires=i)
    for i in range(n_qubits):
        qml.RY(theta_0[i], wires=i)
    for i in range(n_qubits):
        qml.RZ(phi_0[i], wires=i)

    ring_cnot()

    for i in range(n_qubits):
        qml.RY(theta_1[i], wires=i)
    for i in range(n_qubits):
        qml.RZ(phi_1[i], wires=i)

    ring_cnot()

    return qml.probs(wires=range(n_qubits))
```

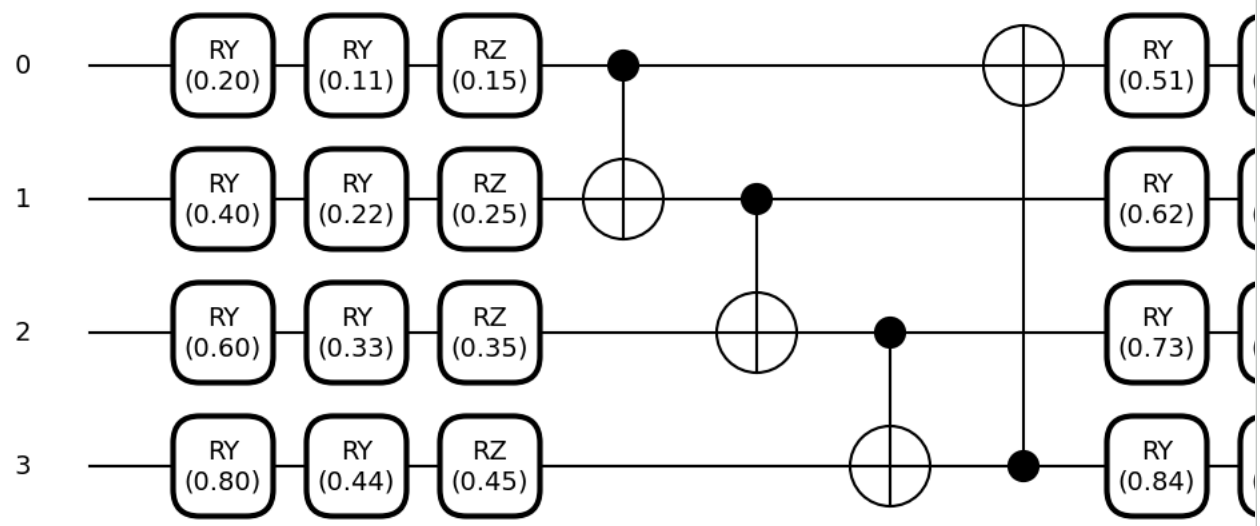
```
probs = circuito_Sebas()
print("Probabilidades:")
print(probs)
print("Suma de probabilidades:", np.sum(probs))
```

```
Probabilidades:
[0.24730297 0.01407562 0.00166658 0.00118146 0.05954204 0.14975303
 0.00347315 0.00475648 0.00392837 0.02720012 0.17215346 0.13493514
 0.0066478  0.01530369 0.11999096 0.03808911]
Suma de probabilidades: 1.0
```

```
print(qml.draw(circuito_Sebas)())
```

```
0: ─RY(0.20)─RY(0.11)─RZ(0.15)─╭●───────╭X─RY(0.51)─RZ(0.18)─╭●───────╭X─┤ ╭Probs
1: ─RY(0.40)─RY(0.22)─RZ(0.25)─╰X─╭●────┤──RY(0.62)─RZ(0.28)─╰X─╭●────┤──┤ ├Probs
2: ─RY(0.60)─RY(0.33)─RZ(0.35)────╰X─╭●─┤──RY(0.73)─RZ(0.38)────╰X─╭●─┤──┤ ├Probs
3: ─RY(0.80)─RY(0.44)─RZ(0.45)───────╰X─╰●─RY(0.84)─RZ(0.48)───────╰X─╰●─┤ ╰Probs
```

```
fig, ax = qml.draw_mpl(circuito_Sebas, decimals=2)()
plt.show()
```




```
estilos_a_probar = ["black_white", "sketch", "pennylane", "solarized_light"]

for estilo in estilos_a_probar:
    fig, ax = qml.draw_mpl(circuito_Sebas, decimals=2, style=estilo)()
    fig.suptitle(f"Estilo: {estilo}", y=1.02)
    plt.show()
```

Mostrar salida oculta

```
import pennylane as qml
from pennylane import numpy as np
import matplotlib.pyplot as plt

print("PennyLane:", qml.__version__)
print("Estilos disponibles:", qml.drawer.available_styles())
```

```
PennyLane: 0.44.1
Estilos disponibles: ('black_white', 'black_white_dark', 'sketch', 'pennylane', 'pennylane_sketch', 'sketch_dark', 'solarized_li
```

```
for estilo in qml.drawer.available_styles():
    fig, ax = qml.draw_mpl(circuito_Sebas, style=estilo)()
    fig.suptitle(f"Estilo: {estilo}")
    plt.show()
```

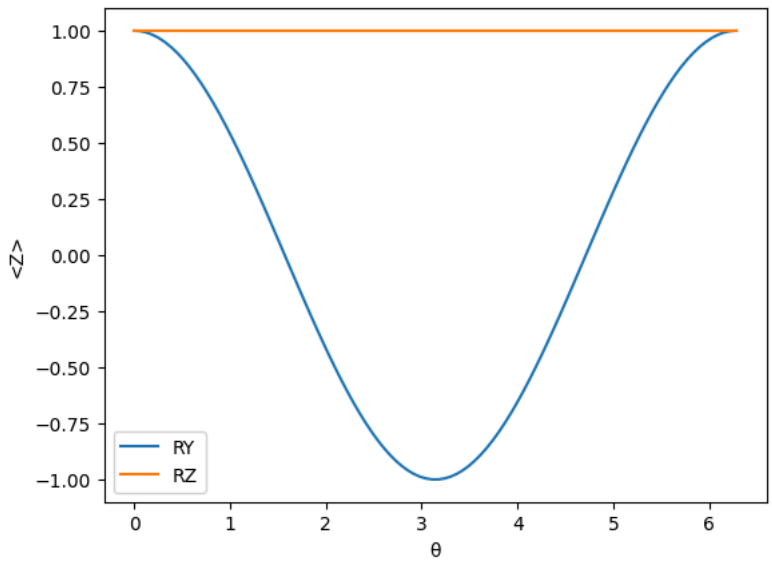
Mostrar salida oculta

C.

```
import pennylane as qml
from pennylane import numpy as np

dev = qml.device("default.qubit", wires=1)

@qml.qnode(dev)
def circuito_RY(theta):
    qml.RY(theta, wires=0)
    return qml.expval(qml.PauliZ(0))

@qml.qnode(dev)
def circuito_RZ(theta):
    qml.RZ(theta, wires=0)
    return qml.expval(qml.PauliZ(0))
```

```
import matplotlib.pyplot as plt

thetas = np.linspace(0, 2*np.pi, 100)

val_RY = [circuito_RY(t) for t in thetas]
val_RZ = [circuito_RZ(t) for t in thetas]

plt.plot(thetas, val_RY, label="RY")
plt.plot(thetas, val_RZ, label="RZ")

plt.xlabel("θ")
plt.ylabel("<Z>")
plt.legend()
plt.show()
```